```
<Policy PolicyId="ConfidentialDocAccess" RuleCombiningAlgId="permit-
overrides">
  <Target>
    <Subjects>
      <AnySubject/>
    </Subjects>
    <Resources>
      <Resource>
        <Attribute AttributeId="resource:type" DataType="string">
          <AttributeValue>confidential-doc</AttributeValue>
        </Attribute>
      </Resource>
    </Resources>
  </Target>

  <Rule RuleId="PermitSameDept" Effect="Permit">
    <Condition>
      <Apply FunctionId="string-equal">
        <AttributeDesignator AttributeId="subject:department"
Category="access-subject" DataType="string"/>
        <AttributeDesignator AttributeId="resource:owner-department"
Category="resource" DataType="string"/>
      </Apply>
    </Condition>
  </Rule>
</Policy>
```

Knowing what you already understand about policies, you can probably read this through several times and develop a sense of its purpose. The `<Target/>` specifies the entities and actions that the policy applies to. The `<Rule/>` defines the conditional access logic. XACML allows a policy to have multiple rules for a given `<Target/>`, and the `RuleCombiningAlgId` from the `<Policy/>` tag indicates how the results of the rules will be combined.

For reference, the equivalent policy in Cedar could be written as follows:

```
permit(principal, action, resource)
when {
    resource.confidential == true &&
    principal.department == resource.owner.department
};
```